README · PROJECT DOCUMENTATION

Education Insights & Resource Recommender

A production-quality multi-agent conversational system built with Google ADK, BigQuery, and Vertex AI Agent Engine. Turns static government school data into actionable answers for parents, educators, and public officials across 30 California schools.

Python 3.11+ | Google ADK 0.5+ | BigQuery | Vertex AI Agent Engine | Gemini 2.0 Flash | 30 CA Schools

TABLE OF CONTENTS

1. Schema Assumptions
2. Project Structure
3. Prerequisites
4. Environment Setup
5. Loading Data into BigQuery
6. Running Locally
7. Deploying to Vertex AI
8. A2A Configuration
9. Running Tests
10. Test Queries & Expected Behaviour
11. Troubleshooting
12. Security & Privacy

SECTION 1

Schema Assumptions

The `sample_data/` files are the source of truth. The following assumptions were made during development and are documented here for transparency.

ASSUMPTION	RATIONALE

`graduation_rate_pct` is NULL for K-5, K-8, and 6-8 schools	CSV confirms empty values for all non-high-school rows
`title_i_funds_usd = 0` (not NULL) when not eligible	All non-eligible schools have `0` in the CSV; NULL would require a different join pattern
`stem_program_level` 1–5 maps to Basic / Moderate / Good / Strong / Exemplary	Confirmed by `programs_technology.csv` text labels in the `stem_program` column
No geographic coordinates are present	Map views require an external geocoding step using school name + district + county + state
Dataset is a static snapshot (no live feed)	Re-run `load_data.py` to refresh BigQuery tables after any CSV update

SECTION 2

Project Structure

```
CapstoneProject/ ├── agents/ │ ├── __init__.py # Exports root_agent as `agent` │ ├── root_agent.py # Entry point; classifies & routes queries │ ├── data_agent.py # BigQuery queries + multimodal extraction │ └── insights_agent.py # Pattern analysis, charts, recommendations ├── tools/ │ ├── bigquery_tools.py # BQ execution helpers (all read-only) │ ├── visualization_tools.py # Plotly chart generators │ └── multimodal_tools.py # Image/PDF extraction via Gemini Vision ├── data/ │ ├── schema.py # BigQuery SchemaField definitions │ ├── queries.py # Parameterized SQL templates (7 queries) │ └── load_data.py # CSV → BigQuery loader ├── deployment/ │ ├── deploy_agents.py # Vertex AI Agent Engine deployment script │ └── a2a_config.py # A2A topology and agent cards ├── tests/ │ ├── test_queries.py # SQL builder + schema tests (offline) │ └── test_agents.py # Agent + tool integration tests (mocked BQ) ├── sample_data/ # 30-school CA dataset (5 CSV files) ├── design_doc/ │ └── architecture.md # Full architecture design document ├── requirements.txt ├── pyproject.toml └── .env.example
```

SECTION 3

Prerequisites

REQUIREMENT	VERSION	NOTES
Python	3.11+	Tested on 3.11 and 3.12

Google Cloud SDK (`gcloud`)	latest	For authentication and API management
GCP project	—	With billing enabled
GCP APIs enabled	—	See command below

Enable required GCP APIs

```
gcloud services enable \
  aiplatform.googleapis.com \
  bigquery.googleapis.com \
  bigquerystorage.googleapis.com \
  storage.googleapis.com
```

SECTION 4

Environment Setup

1 **Navigate to the project directory**

```
cd CapstoneProject
```

2 **Create and activate a virtual environment**

```
# Create
python -m venv .venv

# Activate — Windows
.venv\Scripts\activate

# Activate — macOS / Linux
source .venv/bin/activate
```

3 **Install all dependencies**

```
pip install -r requirements.txt
```

4 **Copy and configure environment variables**

```
cp .env.example .env
```

Edit `.env` with your values:

```
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
BIGQUERY_DATASET=education_insights
GOOGLE_APPLICATION_CREDENTIALS=path/to/service-account-key.json
```

5 **Authenticate (choose one option)**

OPTION A — SERVICE ACCOUNT KEY (LOCAL DEV)

```
export GOOGLE_APPLICATION_CREDENTIALS=path/to/key.json
```

OPTION B — APPLICATION DEFAULT CREDENTIALS (RECOMMENDED)

```
gcloud auth application-default login
gcloud config set project your-project-id
```

SECTION 5

Loading Data into BigQuery

Validate CSVs first (no upload)

```
python -m data.load_data --dry-run
```

Load all five tables

```
python -m data.load_data
```

Refresh — drop and recreate all tables

```
python -m data.load_data --replace
```

Load a single table

```
python -m data.load_data --table staffing
```

Expected output

```
Created dataset your-project.education_insights
Loaded 30 rows → your-project.education_insights.schools_master
Loaded 30 rows → your-project.education_insights.academic_performance
Loaded 30 rows → your-project.education_insights.funding_resources
Loaded 30 rows → your-project.education_insights.staffing
Loaded 30 rows → your-project.education_insights.programs_technology

Done. 150 total rows loaded.
```

SECTION 6

Running Locally (ADK Dev Server)

The ADK CLI provides a web chat UI for local testing before deploying to Vertex AI.

Start the web UI

```
adk web
```

Then open **http://localhost:8000** and select `education_insights_root`.

Run from the command line

```
adk run agents
```

Example queries to try locally

```
1. Identify the five schools in Millbrook County with the highest rate of
   low-income students and the lowest per-pupil technology spending.

2. Find schools with high graduation rates despite below-average funding,
   to discover effective programme models that can be replicated.
```

```
3. Find schools with strong STEM programmes and low class sizes.

4. Compare Summit Ridge High School and Harmony Valley High on graduation
   rates, funding, and teacher retention.

5. Which county has the worst student-to-counselor ratios?
```

SECTION 7

Deploying to Vertex AI Agent Engine

1 **Deploy all agents**

```
python -m deployment.deploy_agents
```

The script prints the resource name on success:

```
Deployed successfully.
Resource name: projects/YOUR_PROJECT/locations/us-central1/reasoningEngines/123456789
```

Add it to `.env`:

```
AGENT_ENGINE_RESOURCE_NAME=projects/YOUR_PROJECT/locations/us-central1/reasoningEngines/123456789
```

2 **Smoke-test the deployment**

```
python -m deployment.deploy_agents --test \
  projects/YOUR_PROJECT/locations/us-central1/reasoningEngines/123456789
```

3 **List all deployments**

```
python -m deployment.deploy_agents --list
```

4 **Delete a deployment**

```
python -m deployment.deploy_agents --delete \
  projects/.../reasoningEngines/123456789
```

> **First deploy takes 5–10 minutes.** Agent Engine packages the code, builds a container, and provisions infrastructure. Subsequent redeploys are faster. If it times out, re-run — the operation is idempotent.

SECTION 8

A2A Configuration

Single-deployment (default)

All three agents — root, data, and insights — are bundled in one Agent Engine deployment. ADK handles inter-agent routing internally via the `sub_agents` parameter. No extra A2A URL configuration is needed.

Cross-deployment A2A (optional)

If you want to deploy the Data Agent independently for reuse by other systems:

```
# Deploy the data_agent as a standalone service
python -m deployment.deploy_agents --name education-data-agent

# Set its resource name
export DATA_AGENT_RESOURCE_NAME=projects/.../reasoningEngines/...
```

Then build a remote reference in code:

```
from deployment.a2a_config import build_remote_agent
remote_data_agent = build_remote_agent("data_agent")
```

See `deployment/a2a_config.py` for the full A2A topology documentation, agent card schemas, and the JSON message contract.

SECTION 9

Running Tests

> **No GCP credentials required.** All tests run offline — BigQuery is mocked with `unittest.mock`.

Run all 30 tests

```
pytest tests/ -v
```

Run only schema and SQL tests

```
pytest tests/test_queries.py -v
```

Run only agent and tool tests

```
pytest tests/test_agents.py -v
```

Run with coverage report

```
pytest tests/ --cov=. --cov-report=term-missing
```

Expected output

```
tests/test_queries.py::TestBuildQuery::test_all_catalog_queries_build PASSED
tests/test_queries.py::TestSchema::test_csv_headers_match_schema[schools_master] PASSED
tests/test_queries.py::TestSchema::test_school_ids_consistent_across_tables PASSED
tests/test_queries.py::TestKnownValues::test_highest_low_income_schools_are_1001_to_1005 PASSED
...
30 passed in 2.34s
```

SECTION 10

Test Queries & Expected Behaviour

QUERY 1 — GRANT PRIORITISATION

> *"Identify the five schools in Millbrook County with the highest rate of low-income students and the lowest per-pupil technology spending."*

EXPECTED BEHAVIOUR

- Routes to `data_agent` → `run_named_query("low_income_tech_gap", county="Millbrook", limit=5)`
- Returns schools 1001–1005 ranked by composite priority score
- Includes caveat that priority_score is a composite index, not an official metric
- Insights Agent adds recommendation to apply for Title I technology grants

EXPECTED TOP RESULT

Riverside Elementary — school_id 1001 — 91.4% low-income — $112.50/pupil tech spending

`1001` `1002` `1003` `1004` `1005`

QUERY 2 — REPLICATION MODELS

> *"Find schools with high graduation rates despite below-average funding, to discover effective programme models."*

EXPECTED BEHAVIOUR

- Routes to `data_agent` → `run_named_query("high_grad_low_funding")`
- Returns schools above state avg graduation rate AND below state avg per-pupil funding
- Insights Agent identifies Summit Ridge High (94.2% grad rate, $8,200/pupil) as top model
- Includes correlation note: "smaller class sizes co-occur with higher graduation rates"
- Explicitly does **not** claim causation

`1018` `1019` `1020` `1021` `1022`

QUERY 3 — STEM + SMALL CLASSES

> *"Find schools with strong STEM programmes and low class sizes."*

EXPECTED BEHAVIOUR

- Routes to `data_agent` → `run_named_query("strong_stem_small_classes", min_stem_level=4, max_class_size=23)`
- Returns schools including 1017 (Elmwood STEM Academy, level 5, avg class 19) and 1028 (Oakdale STEM High)
- Generates bar chart (STEM level by school) and scatter plot (class size vs math proficiency)

`1017` `1022` `1024` `1028` `1029`

QUERY 4 — AMBIGUOUS INPUT (CLARIFICATION)

> *"Which schools need help?"*

EXPECTED BEHAVIOUR

- Root Agent identifies the query as too vague (no metric specified)
- Asks: "Could you be more specific? For example: schools needing help with funding, technology access, staffing, or academic performance?"
- Does **not** proceed to run a query without clarification

QUERY 5 — COUNTY COMPARISON

> *"How do Millbrook County schools compare to Jefferson County schools on per-pupil funding?"*

EXPECTED BEHAVIOUR

- Routes to `data_agent` for data retrieval (`school_comparison` query, county filters)
- Routes to `insights_agent` for analysis
- Returns `county_summary_chart` — mean per-pupil funding by county
- Reports actual county averages and the funding gap between them

SECTION 11

Troubleshooting

PROBLEM	SOLUTION
`GOOGLE_CLOUD_PROJECT not set`	Copy `.env.example` to `.env` and fill in your project ID
`403 Permission denied on BigQuery`	Grant `roles/bigquery.dataViewer` and `roles/bigquery.jobUser` to your service account
`404 Dataset not found`	Run `python -m data.load_data` to create the dataset and load all tables
`ModuleNotFoundError: google.adk`	Run `pip install -r requirements.txt` inside your active virtual environment

`kaleido` errors on chart export	Run `pip install kaleido` or via conda: `conda install -c conda-forge python-kaleido`
Agent Engine deploy times out	First deploy takes 5–10 min; re-run if it times out — Agent Engine is idempotent
`RemoteAgent` not found	Ensure `DATA_AGENT_RESOURCE_NAME` env var is set for cross-deployment A2A
Plotly charts appear blank	Install a display backend: `pip install matplotlib` (used as fallback renderer)

SECTION 12

Security & Privacy Notes

- **Parameterized SQL only.** All BigQuery queries use named parameters (`@param_name`). User input never interpolates into SQL strings — SQL injection is structurally prevented.
- **Read-only enforcement.** The system only reads data. No INSERT, UPDATE, or DELETE is possible through any agent tool. The `run_readonly_sql()` function additionally scans for and blocks DML/DDL keywords.
- **Least privilege IAM.** The service account needs only `roles/bigquery.dataViewer` and `roles/bigquery.jobUser` — no editor or admin roles.
- **Workload Identity in production.** Use Workload Identity instead of service account key files when running on GCP infrastructure.
- **No PII in the dataset.** All records are school-level aggregates. No individual student names, IDs, or personal records are stored or processed.
- **Uploaded documents.** Images and PDFs are sent to the Gemini API for extraction. Do not upload documents containing student PII (names, IDs, addresses). This limitation is noted in the system prompt.
- **FERPA note.** The sample data is synthetic. Any production deployment using real district data must comply with FERPA and applicable state privacy regulations.

> **Contributing:** Install dev dependencies with `pip install pytest pytest-mock ruff`, run the linter with `ruff check .`, and ensure all 30 tests pass with `pytest tests/ -v` before submitting a pull request.